API Security Risk Analysis

Security Assessment & Penetration Testing

Target: VAmPI (Vulnerable API)

GANDONOU Koffi Patrick-Léon

Cyber Security Intern - Future Interns Program

22 March 2026

Table of Contents

Executive Summary **3**

API Analysis Scope & Methodology **4**

Detailed Security Findings **5**

Finding 01: Broken Object Level Authorization (BOLA) 5

Finding 02: Excessive Data Exposure via Debug Routes 7

Finding 03: Weak Password Policy & Rate Limiting 8

Conclusion & Strategic Roadmap **9**

1. Executive Summary

This report documents the findings of a security assessment conducted against the **VAmPI** (Vulnerable API) application, a RESTful web service designed to emulate modern enterprise API architectures. The assessment's objective was to identify systemic vulnerabilities, classify their business risk, and provide actionable remediation strategies.

The analysis uncovered multiple critical flaws directly mapping to the **OWASP API Security Top 10**. The most severe finding is a **Broken Object Level Authorization (BOLA)** vulnerability, which allows any authenticated user to maliciously exfiltrate data belonging to any other user in the database. Furthermore, administrative debugging routes were left exposed in production, leading to severe information disclosure.

> **Business Impact:** If this API were deployed in a production environment, an attacker could harvest the entirety of the application's user base and sensitive data, leading to a massive data breach, regulatory fines (GDPR/CCPA), and loss of customer trust.

2. API Analysis Scope & Methodology

Tools Leveraged

- Postman: For manual endpoint testing, token management, and payload crafting
- Python (Requests): For developing an automated exploit script to validate the BOLA vulnerability programmatically
- Docker: Local isolated hosting of the target API (erev0s/vampi)

Endpoints Assessed

Endpoint	Purpose
`POST /users/v1/register`	Account creation
`POST /users/v1/login`	Authentication & JWT Issuance
`GET /users/v1/{username}`	Data retrieval

Endpoint	Purpose
`GET /users/v1/_debug`	Administrative diagnostics

3. Detailed Security Findings

FINDING 01: Broken Object Level Authorization (BOLA)

Risk Severity	**CRITICAL**
OWASP Category	API1:2023 - Broken Object Level Authorization
Affected Endpoint	GET /users/v1/{username}

Technical Description

When a user logs in, the API issues a JSON Web Token (JWT). However, when requesting user data via the /users/v1/{username} endpoint, the API only verifies that the JWT is valid, but fails to verify if the user identified by the JWT is the actual owner of the data being requested.

Proof of Concept (Exploit)

Using our custom script (vampi_exploit.py), we logged in as a standard user (hacker1). We then injected the administrator's username into the URL path:

```
GET /users/v1/admin
```

Response received:

```
{
    "username": "admin",
    "email": "admin@mail.com"
}
```

The API leaked the administrator's private data to a low-level user.

Business Risk (Simple Language)

Imagine a hotel where your electronic room key (the JWT) correctly proves you are a paying guest, but the hotel doors don't check which room you paid for. You can open any door in

the building. Hackers use this flaw to scrape millions of accounts by automating requests with sequential IDs or usernames.

Remediation

- Code Level: Implement strict Authorization checks immediately after Authentication. The application must verify: if requested_username != username_in_jwt: return 403 Forbidden
- Design Level: Replace guessable identifiers (like usernames or sequential integers) with non-predictable UUIDv4 (Universally Unique Identifiers) in API routes

FINDING 02: Excessive Data Exposure via Debug Routes

Risk Severity	**HIGH**
OWASP Category	API5:2023 - Broken Function Level Authorization
Affected Endpoint	GET /users/v1/_debug

Technical Description

The API exposes a diagnostic endpoint (_debug) containing detailed information about every user in the system, system properties, and application configuration. This endpoint is publicly accessible without requiring an administrative JWT.

Business Risk (Simple Language)

Developers often create hidden "backdoors" or debug pages to make troubleshooting easier during development. When these pages are accidentally pushed to the live server, they provide hackers with a goldmine of insider knowledge, making it drastically easier to plan a full system compromise.

Remediation

- Disable or completely remove all debug and diagnostic endpoints in production builds
- If administrative monitoring is required, enforce strict Role-Based Access Control (RBAC) ensuring only tokens with an "admin": true claim can access the route

FINDING 03: Weak Password Policy & Rate Limiting Absence

Risk Severity	**MEDIUM**
OWASP Category	API2:2023 - Broken Authentication
Affected Endpoint	POST /users/v1/login

Technical Description

The authentication endpoint does not enforce rate limiting (throttling). Furthermore, the API permits registration of accounts with trivial passwords.

Business Risk (Simple Language)

A hacker can write a script to guess thousands of passwords per second (Brute Force Attack) against a user's account. Because the API never slows down or blocks the attacker after multiple failed attempts, it is only a matter of time before weak accounts are cracked.

Remediation

- Implement API Rate Limiting (e.g., maximum 5 login attempts per minute per IP address)
- Enforce strict password complexity during the /register phase (Minimum 10 characters, mixed case, special symbols)

4. Conclusion & Strategic Roadmap

The VAmPI application serves as a perfect demonstration of why API security differs from traditional web security. Firewalls and WAFs cannot detect a BOLA attack because the HTTP request looks completely legitimate - the flaw lies purely in the application's business logic.

To secure this API for production, the engineering team must adopt a **Zero Trust** approach to authorization. Every single endpoint that returns or modifies data must contextually verify the privileges of the token requesting the action.

Remediation Priority Matrix

Priority	Finding	Timeline
P1	BOLA Authorization Bypass	Immediate (24-48h)
P2	Debug Route Exposure	Short-term (1 week)
P3	Rate Limiting & Passwords	Medium-term (2-4 weeks)

API Security Risk Analysis

Security Assessment & Penetration Testing

GANDONOU Koffi Patrick-Léon

Cyber Security Intern - Future Interns Program

© 2026 - Internal Assessment